

# An authorization firewall for AI agents

## From product thesis to a working decision service

Prepared for Suraj Patel | September 13, 2026

A practical design for controlling agent actions, with architecture diagrams, policy rules, failure handling, and an eight-step implementation tutorial.

### How to use this paper

Read Part I to understand the system and its security boundaries. Follow Part II in order to build the local exercise, then use the checkpoints to develop a durable MVP. References link to primary technical sources.

Status: proposed system, not a peer-reviewed empirical study. No production security, customer traction, or measured performance is claimed. The tutorial begins with mock facts and no live financial operations.

## Part I. Research and system design

### Abstract

An AI agent can produce a plausible plan while attempting an unauthorized action. Once it has credentials to issue refunds, send messages, or change infrastructure, instructions inside a prompt are an insufficient enforcement boundary. This paper proposes an authorization firewall: a service that receives structured action requests, checks independently verified identity and business policy, obtains approval when required, and executes permitted operations through controlled connectors. The central thesis is that useful autonomy requires authority to be enforced outside the model. A practical first product is a TypeScript SDK, Fastify gateway, deterministic policy evaluator, durable execution worker, and approval dashboard for customer-support refunds. This paper specifies the trust model, action lifecycle, data model, build sequence, and evaluation criteria. Its proposed contribution is a cohesive execution boundary, rather than a new prompt-injection classifier.

## 1. Problem and product thesis

The product promise is straightforward: **allow companies to delegate bounded actions to agents while retaining control over what actually executes**. "Cloudflare for AI actions" is a useful positioning analogy, but the engineering resembles an authorization gateway with a credential broker and durable workflow engine.

OWASP's agentic-security guidance identifies goal hijacking, tool misuse, identity and privilege abuse, and memory poisoning among the risks facing agent systems [1, 2]. An authorization firewall addresses the consequences of some of these failures. It does not establish that an agent's beliefs, memories, or reasoning are correct.

Consider a support agent reading a customer ticket containing instructions to ignore refund limits. The firewall should enforce the same refund policy whether the request came from a helpful agent, a confused agent, or an agent influenced by that ticket. It can verify the proposed operation and its parameters; it cannot know the model's true intent.

The initial customer hypothesis is an AI startup or medium-sized software company that already has a support agent and is reluctant to enable write access. The economic buyer may be the engineering lead responsible for that workflow. The first discovery question is concrete: "Which action are you currently keeping manual because you cannot safely delegate it?" A documented security risk establishes relevance, not willingness to pay.

## 2. Start with one enforceable workflow

Build support refunds first. Offer three typed operations: retrieve limited order information, request a refund, and inspect the request's status. Do not initially expose arbitrary HTTP requests, SQL, shell commands, bank-account modifications, or deployments.

Use the following explicit policy. The original idea leaves exactly \$50 unspecified; this blueprint conservatively requires approval at **\$50 or more**.

| Condition                                                         | Outcome                               |
|-------------------------------------------------------------------|---------------------------------------|
| Unknown tool, invalid input, suspended agent, or wrong tenant     | Deny                                  |
| Any destination-bank-account change                               | Deny; no connector operation provided |
| Refund of 1-4,999 USD cents with all other checks satisfied       | Allow                                 |
| Refund of 5,000 USD cents or more with all other checks satisfied | Manager approval                      |
| Refund exceeds remaining refundable amount                        | Deny                                  |
| Aggregate limit would be exceeded                                 | Deny, including after approval        |

An illustrative tenant limit is \$500 per UTC day across its agents. This is a configurable design assumption, not an industry standard. Also track cumulative refunds per order: ten small requests must not circumvent the intended approval threshold. For this MVP, require approval when the order's committed, reserved, and proposed refund total reaches \$50. Limit pending requests to prevent approval-queue flooding.

Refunds must return through the provider's original-payment refund operation. A manager can satisfy an approval requirement but cannot override a hard denial. Start in a provider sandbox with synthetic customer records.

### 3. Architecture and trust boundary

The SDK provides integration convenience. **The gateway and connector are the enforcement boundary.** Agents must not retain the downstream credentials that would let them bypass it. In customer-controlled deployments, restrict the agent runtime's network access as well. If a customer keeps alternate credentials or unrestricted privileged tools, describe that integration as partial coverage.

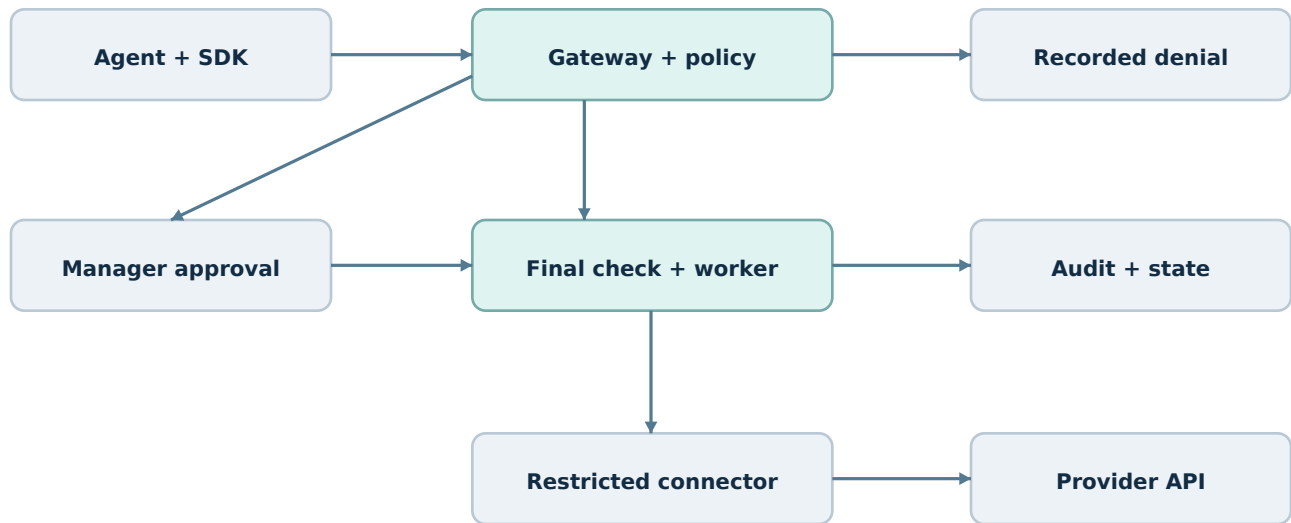


Figure 1. Every supported write passes through the controlled connector. Approval is an input to final authorization, not permission to bypass it.

Use TypeScript for shared action types and Fastify for request handling. Fastify supports JSON Schema validation and warns that schemas should be treated as application code [3]. Ship reviewed schemas with connectors; never compile an agent-supplied schema. Reject unexpected fields and unsupported content types, cap payload sizes, and perform database-backed checks after structural validation.

Use PostgreSQL for policies, action state, approvals, reservations, and a transactional outbox. A worker consumes that outbox; a separate message broker is unnecessary for the first version. Build a small React dashboard for approvals, action history, agent suspension, and policy publication. Keep provider secrets in a managed secret store, accessible only to the execution service. Use distinct sandbox and production credentials.

The trusted computing base includes the gateway, worker, connector code, identity service, policy administration, database, and secret store. Assume ticket content, model output, supplied reason strings, and tool responses may be hostile. Compromise of the enforcement service or its privileged administrator is outside the MVP's prevention guarantee; hardening that boundary remains essential.

### 4. Identity, action contracts, and policy evaluation

Authenticate each workload with a short-lived credential identifying a registered agent. Resolve tenant membership and agent status on the server. Where an agent acts on behalf of a user, constrain effective authority to the intersection of tenant policy, agent permissions, delegated user permissions, and connector scope. A request body must not be able to nominate its own tenant or manager role.

Expose this conceptual SDK contract:

```
// Proposed interface, not an existing package or runnable implementation.
const action = await firewall.request({
  tool: "refund.create",
  arguments: {
    orderId: "ord_demo_123",
    amountMinor: 4900,
    currency: "USD"
  },
  idempotencyKey: "support-case-123-refund-v1",
  reason: "Customer reported a duplicate purchase."
});
// Returns actionId and a durable status; never executes locally.
```

Validate `amountMinor` as a positive safe integer. Accept only USD in version one. Retrieve tenant ownership, payment reference, remaining refundable amount, and original payment destination from authoritative records. Treat `reason` as untrusted explanatory text. It must never grant authority.

The policy evaluator returns `ALLOW`, `DENY`, or `REQUIRE_APPROVAL`, plus stable reason codes and a policy version. Evaluation errors and missing required facts produce denial. Make explicit-deny precedence part of the specification. Open Policy Agent documents default-deny behavior [4]; a small, typed TypeScript evaluator is sufficient for this narrow MVP. Preserve its interface so a policy engine can replace it later. Do not execute customer-authored JavaScript as policy.

The authorization predicate is:

Execute only if identity is valid, agent is active, the operation is allowed, authoritative facts satisfy policy, required approval is valid, capacity is reserved, and the action is eligible for dispatch.

Persist a canonical request representation and its hash. Bind approvals to that hash, tenant, agent, connector version, policy version, and an expiry. Changing any action parameter creates a new request. A policy change invalidates outstanding approvals in this first version; reevaluate and request approval again where necessary.

## 5. Durable approvals and safe execution

An approval must survive browser reloads and service restarts. It should identify exactly what the manager authorized and expire after a configurable interval, initially 15 minutes. The dashboard shows the amount, customer/order reference, cumulative exposure, applicable policy, expiry, and the agent's separately labeled explanation. Escape untrusted text, require authenticated manager access, prevent self-approval, and protect browser mutations against cross-site request forgery.

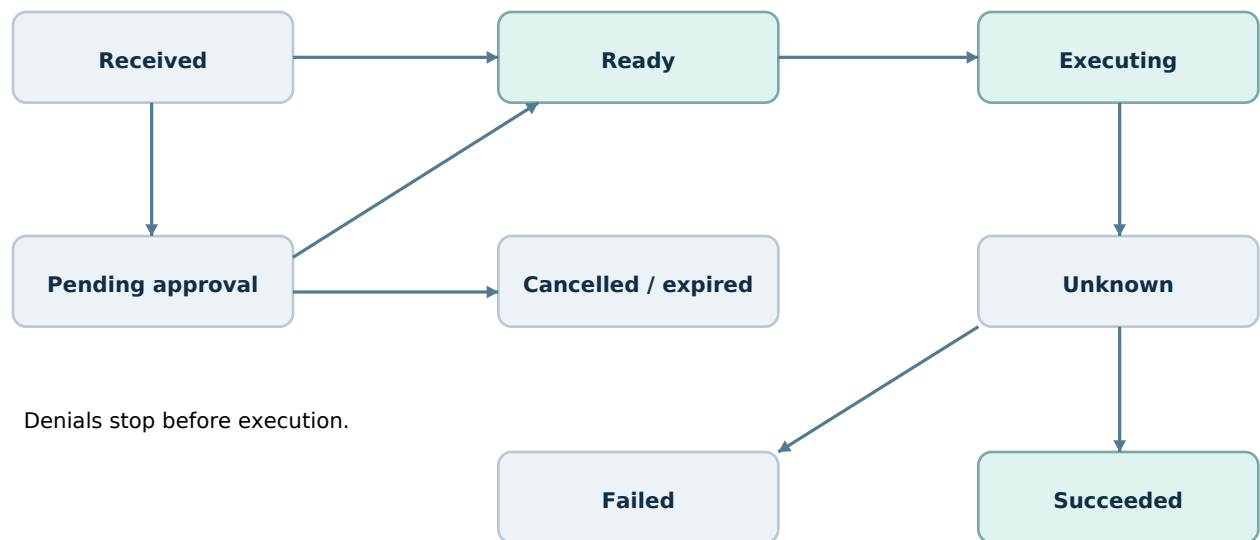


Figure 2. Simplified lifecycle: definitive responses transition directly from Executing to Succeeded or Failed. An ambiguous provider response is a distinct state. It must not be interpreted as permission to submit another refund.

Within a database transaction, lock the relevant tenant budget and order reservation records, reevaluate current policy, verify approval, reserve capacity, and record the execution attempt. Concurrent requests must compete for the same locked capacity. Keep reservations for unknown outcomes until reconciliation resolves them; release them only on definitive failure or cancellation before dispatch. Use a consistent lock order and bounded retries to handle database contention.

There is no atomic transaction spanning PostgreSQL and an external provider. Persist a stable provider idempotency key before sending the request, and reuse it on supported retries. Stripe documents that idempotency keys return the saved result for repeat requests and may be removed after at least 24 hours [5]. Maintain a durable local deduplication record beyond that provider window. Never blindly retry an unresolved request after the provider's protection may have expired.

Enforce a unique `(tenant_id, idempotency_key)` constraint; reject reuse with different parameters. A worker claim prevents parallel local execution, while provider idempotency handles a crash after dispatch but before recording success. Reconcile ambiguous results with provider records or verified webhooks. Do not promise universal exactly-once execution across arbitrary tools.

External changes can also invalidate local facts. Provider-side refundable-balance checks remain necessary. Where a future tool cannot condition a write on a resource version or otherwise enforce a critical precondition, disclose the race or exclude that operation.

## Emergency stop semantics

Support suspension at tenant and agent level. Serialize suspension and dispatch authorization against the same authoritative state record; workers must not use stale cached permission. Suspension committed before dispatch authorization blocks that dispatch. An action already authorized or sent may complete. Cancel queued work, invalidate outstanding approvals, and record in-flight requests for reconciliation. The kill switch prevents future dispatch authorizations; it does not undo a completed refund. Measure the remaining authorization-to-send interval rather than promising instantaneous cancellation.

## 6. Prompt injection and the audit trail

The primary injection defense is a narrow action vocabulary plus independent authorization. A poisoned ticket can request an arbitrary bank change, but the connector has no operation to express it. It can request a large refund, but the trusted policy requires approval. It may still induce a harmful refund that falls within policy. That residual risk is why authoritative business checks, aggregate limits, and clear human review matter.

Optional injection detection can flag suspicious text or escalate a request; it must not override denial or become the source of permission. Add detectors only after establishing a labeled evaluation set and measuring false positives. Scope returned order data to necessary fields and enforce read permissions too: authorized writes do not prevent information leakage through reads or other channels.

Record a **decision and action trail**, not a claim to complete internal model reasoning. Store the normalized request, verified actor, relevant fact snapshot, policy version, reason codes, approval identity, timestamps, execution attempts, provider reference, and final result. An agent-provided rationale is an unverified explanation, not evidence of its internal reasoning.

Redact credentials and unnecessary customer information. Use tenant-scoped access and retention settings. Give the application an insert-only audit role where feasible; an append-only table alone does not resist a database administrator. Later, export signed audit batches to independently controlled immutable storage. Log policy publication and suspension with the same care as actions.

## 7. Implementation structure and API

Organize the repository around explicit responsibilities:

| Path                | Responsibility                                              |
|---------------------|-------------------------------------------------------------|
| apps/gateway        | Authentication, validation, action submission and status    |
| apps/worker         | Final checks, reservations, connector calls, reconciliation |
| apps/dashboard      | Approval queue, policies, audit history, emergency stop     |
| packages/contracts  | Reviewed schemas, action types, reason codes                |
| packages/policy     | Deterministic evaluator and policy fixtures                 |
| packages/connectors | Mock refund provider, then sandbox integration              |
| packages/sdk        | Submission, polling, typed errors, retry behavior           |

The minimum tables are tenants, agents, policy\_versions, actions, approvals, budget\_reservations, execution\_attempts, outbox, and audit\_events. Every tenant-owned record carries tenant\_id; queries and composite foreign keys must prevent cross-tenant associations. Add row-level security as defense in depth. Policy versions and submitted requests are immutable; state transitions use transactions and compare-and-set checks.

| Endpoint                        | Behavior                                               |
|---------------------------------|--------------------------------------------------------|
| POST /v1/actions                | Validate, decide, persist; return action ID and status |
| GET /v1/actions/:id             | Return authorized tenant's action status               |
| POST /v1/approvals/:id/decision | Manager approves or rejects exact request              |
| POST /v1/agents/:id/suspend     | Suspend agent and invalidate pending authority         |
| POST /v1/tenant/suspend         | Stop dispatch authorization tenant-wide                |
| POST /v1/policies               | Validate and publish a new immutable policy version    |

A successful submission response means the action was recorded, not that the provider completed it. The SDK polls status with backoff. Defer public webhooks and their additional delivery/security work until a pilot needs them.

Add an MCP adapter after the HTTP path works. MCP's HTTP authorization specification addresses transport access, including audience validation and prohibiting token passthrough [6]. It does not replace refund-specific business policy. Use separate downstream credentials and map every exposed MCP tool into the same action lifecycle.

## 8. A six-week build sequence

This is a planning estimate for focused development, not a production-readiness guarantee. Part-time work alongside college will likely take longer. Your TypeScript/Fastify and API-workflow experience from UniversalAPI transfers directly to the gateway, schema contracts, and monitoring portions.

| Stage               | Deliverable                                                            | Exit criterion                                                                 |
|---------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Week 1: boundary    | Monorepo, database migrations, agent identity, mock connector          | A cross-tenant or unknown-tool request never reaches the connector             |
| Week 2: policy      | Refund rules, immutable requests, reason codes, aggregate reservations | Boundary amounts and concurrent limit tests behave exactly as specified        |
| Week 3: approval    | Dashboard, authenticated manager decisions, expiry and revalidation    | Changed or expired requests cannot reuse approval                              |
| Week 4: execution   | Outbox worker, idempotency, ambiguous-state reconciliation, suspension | Crash/retry tests produce no duplicate provider side effect                    |
| Week 5: integration | Refund sandbox connector, SDK, deployment, audit filtering             | Full sandbox allow/approve/deny workflow runs from an example agent            |
| Week 6: pilot       | Shadow evaluation, then restricted enforcement with a design partner   | Partner can install, explain a decision, approve an action, and stop execution |

Begin with three fixtures: \$49 allowed, \$50 awaiting approval, and destination change denied. Then demonstrate two concurrent refunds competing for the last available budget and a worker crash immediately after a provider accepts a refund. These tests establish more value than a polished landing page.

Deploy one gateway, one worker, and managed PostgreSQL in the same region for the pilot. Separate application and migration privileges, use TLS and secret rotation, and test backup restoration. If identity, policy, or durable audit storage is unavailable, fail closed for writes. Do not advertise a production uptime commitment until recovery behavior is measured.

## 9. Evaluation plan and technical measurements

The research question is whether independent action enforcement reduces unauthorized side effects without making legitimate workflows unusable. Compare three configurations against the same labeled requests: prompt-only instructions, an SDK precheck with an intentionally available direct credential, and the enforced gateway with credentials removed from the agent. Perform bypass experiments only against the mock provider or sandbox.

| Test family                                                   | Required observation                                         |
|---------------------------------------------------------------|--------------------------------------------------------------|
| Amounts: 0, 4,999, 5,000 cents; invalid currency or fractions | Correct validation and threshold decision                    |
| Wrong tenant, forged role, unknown tool                       | Zero connector dispatches                                    |
| Changed arguments, expired approval, policy update            | No stale approval execution                                  |
| Parallel requests and split refunds                           | Aggregate reservations preserve configured limits            |
| Worker crash, timeout, duplicate request                      | Stable key and reconciliation prevent duplicate side effects |
| Suspension during queued execution                            | No dispatch authorization after suspension commits           |
| Poisoned ticket and convincing agent explanation              | Neither can override a deterministic denial                  |

Report unauthorized execution rate, legitimate denial rate, approval frequency, duplicate side effects, and p50/p95/p99 gateway overhead. Count actual connector side effects, not merely API denial responses. Report scenario counts and failures; zero failures in a finite test set is not proof of universal security.

The latency model is:

total time = gateway decision + queue wait + provider time + human wait (when required)

Set a provisional target of p95 gateway overhead below 50 ms at 100 requests/second for same-region, cached-independent-fact tests; separately benchmark uncached authoritative lookups. These are targets, not measured results. Exercise sustained load and concurrent writes, publish machine/database configuration, and report tail latency and error rate. Human approval should be asynchronous and excluded from the machine-only latency metric.

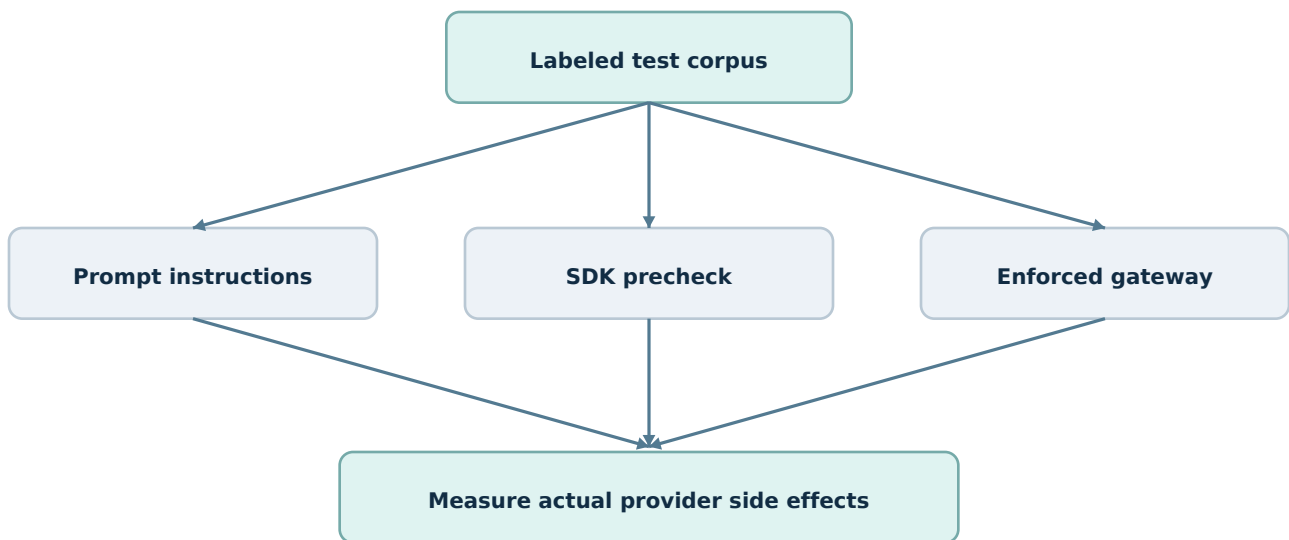


Figure 3. Proposed experimental graph. The paper includes no fabricated performance chart; numerical plots should be generated from this experiment's recorded results.

## 10. Commercialization and defensibility

Treat \$99-\$499 per month as a pricing hypothesis. A possible \$99 tier includes a limited action allowance and one environment; a \$499 tier adds higher usage, multiple environments, and longer audit retention. Define fair-use limits and price expensive integrations separately. Estimate contribution margin as revenue minus hosting, database, storage, third-party processing, and support. Mandatory model calls on every action could add cost and latency without strengthening deterministic enforcement.

Interview five to ten teams already shipping action-taking agents. Ask for the blocked workflow, present approval process, incident history, integration constraints, and purchasing owner. Seek two design partners willing to route one real workflow through a controlled pilot. A proposed continuation gate is one paid pilot plus a second committed deployment after the initial validation cycle; treat it as a founder decision rule, not evidence of market demand.

The basic proxy and policy evaluator are reproducible. Defensibility would need to come from reliable connector semantics, approval usability, integration speed, operational trust, and a growing regression suite of failure cases. Compare against customers' existing IAM, gateway, and workflow tooling during interviews. This paper does not claim an empty competitive category or an established moat.

## 11. Conclusion

The strongest first version is a constrained refund authorization service with a clear execution boundary. Its value is that a business rule survives prompt injection, model confusion, retries, concurrent requests, and human-review delays. Build deterministic permission checks, immutable approvals, credential isolation, and durable execution before adding broad tool coverage or probabilistic detection. A successful prototype should demonstrate one simple outcome: the agent can propose an action, but only independently authorized actions reach the provider.

## Part II. Guided implementation tutorial

This tutorial takes you from an empty folder to a local authorization decision service, then explains how to add the components that turn it into the proposed firewall. Follow the steps in order. Each step names what to build, why it matters, and the result you should see before continuing.

The first exercise deliberately does not move money. It evaluates mock refunds using fixed server-side facts. It is complete as a learning exercise, but it does not implement the full research design. Do not connect live credentials until the durable execution, identity, approval, and isolation milestones are complete.

### Step 1. Create the project

Use a supported Node.js LTS release compatible with Fastify 5, npm, and your editor. Check `node --version` and `npm --version`. Run the following in your terminal. Keep the generated lockfile in version control so subsequent installations are reproducible.

```
mkdir agent-firewall
cd agent-firewall
npm init -y
npm pkg set type=module
npm install fastify@5
npm install -D typescript tsx @types/node
mkdir src
npm pkg set 'scripts.dev=tsx src/server.ts'
npm pkg set 'scripts.test=tsx --test src/policy.test.ts'
```

Create the three files in the next steps inside `src`. The development runner executes TypeScript; it does not replace static type checking. Add a strict TypeScript configuration and a `tsc --noEmit` check before expanding this into a larger application.

**Checkpoint:** `npm ls fastify` should show an installed Fastify 5 release. If npm cannot find a package or reports an incompatible Node version, fix that environment issue before writing application logic. Fastify's official getting-started guide explains installation and ESM setup [7].

### Step 2. Write a pure policy function

Create `src/policy.ts` with the following code. A pure function returns a decision from supplied facts without calling a model, database, or payment service. This makes the rules understandable and testable. The gateway will supply these facts; the caller must never be trusted to set them.

```

export type Facts = {
  tool: string;
  amountMinor: number;
  currency: string;
  active: boolean;
  remainingMinor: number;
  orderExposureMinor: number;
  budgetAvailableMinor: number;
};

export function decide(f: Facts) {
  const deny = (reason: string) => ({
    decision: 'DENY', reason
  });
  if (!f.active) return deny('AGENT_SUSPENDED');
  if (f.tool !== 'refund.create')
    return deny('TOOL_NOT_ALLOWED');
  if (!Number.isSafeInteger(f.amountMinor) ||
    f.amountMinor < 1 || f.currency !== 'USD')
    return deny('INVALID_AMOUNT_OR_CURRENCY');
  if (f.amountMinor > f.remainingMinor)
    return deny('EXCEEDS_REFUNDABLE_BALANCE');
  if (f.amountMinor > f.budgetAvailableMinor)
    return deny('BUDGET_EXCEEDED');
  if (f.orderExposureMinor + f.amountMinor >= 5000)
    return {
      decision: 'REQUIRE_APPROVAL',
      reason: 'ORDER_EXPOSURE_THRESHOLD'
    };
  return { decision: 'ALLOW', reason: 'WITHIN_POLICY' };
}

```

The amount uses cents to avoid floating-point dollar arithmetic. The function checks hard denials before approval. Approval therefore cannot rescue an over-budget request. `orderExposureMinor` represents previous committed refunds plus active reservations; it is not merely the amount in this request. Validate all loaded facts at the service boundary and deny when a required fact is unavailable.

### Step 3. Add the local gateway

Create `src/server.ts`. This teaching service binds only to your own computer. It uses a known mock order and no provider credentials. The endpoint returns a decision; an `ALLOW` response is not a completed refund.

```

import Fastify from 'fastify';
import { decide } from './policy.js';

const app = Fastify({
  logger: true,
  ajv: { customOptions: {
    removeAdditional: false,
    coerceTypes: false
  } }
});

type Input = {
  tool: string;
  orderId: string;
  amountMinor: number;
  currency: string;
};

app.post<{ Body: Input }>('/v1/decisions', {
  schema: { body: {
    type: 'object',
    additionalProperties: false,
    required: ['tool', 'orderId',
      'amountMinor', 'currency'],
    properties: {
      tool: { type: 'string', maxLength: 64 },
      orderId: { type: 'string', maxLength: 64 },
      amountMinor: { type: 'integer', minimum: 1,
        maximum: 9007199254740991 },
      currency: { type: 'string', enum: ['USD'] }
    }
  } }
}, async (request) => {
  const b = request.body;
  if (b.orderId !== 'ord_demo_123')
    return { decision: 'DENY', reason: 'ORDER_NOT_FOUND' };
  return decide({
    tool: b.tool,
    amountMinor: b.amountMinor,
    currency: b.currency,
    active: true,
    remainingMinor: 20000,
    orderExposureMinor: 0,
    budgetAvailableMinor: 50000
  });
});

await app.listen({ port: 3000, host: '127.0.0.1' });

```

**Checkpoint:** Run `npm run dev`. Fastify should report a listening address on port 3000. If the port is occupied, stop the other local service or change the port in both this file and the next command. The explicit validation settings prevent silent removal of extra fields and coercion of strings into numbers.

## Step 4. Exercise the three decisions

In a second terminal, send this request while the server remains running:

```

curl -s http://127.0.0.1:3000/v1/decisions \
  -H 'Content-Type: application/json' \
  -d '{"tool": "refund.create", "orderId": "ord_demo_123",
    "amountMinor": 4900, "currency": "USD"}'

```

Expect ALLOW and WITHIN\_POLICY. Change 4900 to 5000 and expect REQUIRE\_APPROVAL. Change the tool to bank.update and expect DENY. Add a manager: true field and expect HTTP 400, because the schema rejects undeclared fields. The HTTP status describes request handling; the decision describes authorization.

Create src/policy.test.ts to verify the boundary without starting the server. Node's test runner supports assertions and test cases [8].

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { decide, type Facts } from './policy.js';

const base: Facts = {
  tool: 'refund.create', amountMinor: 4900,
  currency: 'USD', active: true,
  remainingMinor: 20000, orderExposureMinor: 0,
  budgetAvailableMinor: 50000
};

test('refund policy boundaries', () => {
  assert.equal(decide(base).decision, 'ALLOW');
  assert.equal(decide({ ...base, amountMinor: 5000 })
    .decision, 'REQUIRE_APPROVAL');
  assert.equal(decide({ ...base, tool: 'bank.update' })
    .decision, 'DENY');
  assert.equal(decide({ ...base, active: false })
    .decision, 'DENY');
  assert.equal(decide({ ...base, budgetAvailableMinor: 1 })
    .decision, 'DENY');
  assert.equal(decide({ ...base, orderExposureMinor: 100 })
    .decision, 'REQUIRE_APPROVAL');
});
```

**Checkpoint:** Run `npm test`; the test should pass. You now have a decision service with repeatable checks. The fixed facts mean this exercise does not accumulate spending across requests and cannot safely authorize real execution. The next steps replace that limitation systematically.

## Step 5. Replace fixtures with authenticated, durable actions

Install a PostgreSQL driver and migration tool of your choice. Create the tables described in Part I. Implement POST /v1/actions as the durable counterpart to the teaching endpoint. Authenticate the workload, derive its tenant server-side, load the order within that tenant, normalize parameters, and calculate a canonical request hash. Add a unique constraint on tenant plus idempotency key.

In one transaction, create the immutable action, its decision event, and either an approval request or an outbox entry. Reusing a key with the same hash returns the original action; reusing it with another hash returns a conflict. Do not let the agent select the provider account or the tenant through request fields.

**Checkpoint:** Submit one action twice, restart the server, and submit it again. All responses must identify the same action. A second tenant must be unable to fetch it. Fail the transaction deliberately and verify that neither a partial action nor a dispatchable outbox entry survives.

## Step 6. Implement approval as a state transition

Create a manager-only approval endpoint and a dashboard with Pending, Completed, and Expired views. Show the exact order, amount, aggregate exposure, policy reason, and expiry. Start with approve/reject buttons; editing an approved amount must create a new request. On approval, lock the action and verify tenant, manager role, request hash, policy version, expiry, and pending status before recording the decision and enqueueing work.

**Checkpoint:** Approve the \$50 fixture and see it become ready for execution. Repeat the approval and confirm it does not enqueue another logical action. Expire an approval or publish a new policy and verify the old approval is unusable. An agent credential must receive an authorization error from the manager endpoint.

## Step 7. Build the worker before connecting a provider

Implement a mock connector with a durable side-effect ledger keyed by provider idempotency key. The worker claims an outbox item, obtains current authorization, locks order and budget records, and reserves the amount before dispatch. Persist the attempt and stable provider key before calling the connector. Keep the network call outside a long database transaction; recovery is handled through the state machine and idempotency.

Inject a crash after the mock connector records success but before the worker records completion. Restart the worker and reconcile or retry using the same key. The ledger must still contain one refund. Do not release an unknown outcome's reservation. This exercise is the central reliability checkpoint for the product.

**Checkpoint:** Simultaneously submit two \$40 requests when only \$50 remains. At most one may reserve and dispatch. Then suspend the agent with work queued and verify no new dispatch authorization occurs after suspension commits. Record already-authorized in-flight work separately.

## Step 8. Add the sandbox connector and pilot dashboard

Replace the mock adapter with a reviewed refund-provider sandbox adapter. Map only the typed refund operation, obtain the payment reference from tenant-owned records, and keep the provider secret exclusively in the worker's secret store. Use the provider's documented idempotency and reconciliation semantics. Do not turn the connector into a generic URL relay.

Add status polling to the SDK and link each dashboard action to its approval, policy version, provider reference, and audit events. Introduce response-field filtering for order reads. Run the failure matrix in Part I against the sandbox, including provider timeouts, revoked identity, expired approval, split refunds, and suspension.

**Checkpoint:** A user can submit a small refund, approve a larger one, inspect why a prohibited request was denied, and suspend the agent. A direct privileged call from the agent runtime must fail because the credential and execution path are absent. Only after these checks should a design partner consider a tightly limited live pilot.

## How to work through the tutorial

Spend your first session completing Steps 1-4. In the next sessions, implement one durable transition at a time: submission, approval, dispatch, reconciliation, then suspension. Commit each working milestone. Keep a short decision log for policy boundaries and unresolved connector behavior. Avoid broadening the tool catalog until the refund workflow passes the crash and concurrency tests.

# References

Primary sources checked September 13, 2026. Architecture, thresholds, schedule, pricing, and evaluation targets are proposals in this paper, not claims made by these sources.

1. OWASP GenAI Security Project. [OWASP Top 10 for Agentic Applications for 2026](#). December 9, 2025.
2. OWASP GenAI Security Project. [The Benchmark for Agentic Security in the Age of Autonomous AI](#). December 9, 2025.
3. Fastify. [Validation and Serialization](#). Official documentation.
4. Open Policy Agent. [Rego keyword: default](#). Official documentation.
5. Stripe. [Idempotent requests](#). Official API reference.
6. Model Context Protocol. [Authorization, specification version 2025-11-25](#). Versioned reference, not a claim that this is the latest specification.
7. Fastify. [Getting Started](#). Official documentation.
8. Node.js. [Test runner](#). Official documentation.